

FlipTech

Rapport de développement

PROJET INFORMATIQUE 2 – PREING1

CY Tech

Développement d'un jeu de cartes stratégique en langage C

Brahim BENALI

Kais KHEMIRI

Youssef FARSAN

Année universitaire 2025–2026

22 mai 2026

Table des matières

1	Introduction	2
2	Architecture logicielle	2
2.1	Organisation du programme	2
2.2	Gestion mémoire	3
2.3	Refactorisation du code	3
3	Organisation du travail	3
4	Détails d'implémentation	4
4.1	Module Deck	4
4.2	Module Rules	4
4.3	Interface utilisateur	4
4.4	Système de sauvegarde	5
5	Difficultés rencontrées	5
5.1	Gestion des cas limites	5
5.2	Gestion des entrées utilisateur	5
5.3	Fonctionnalités abandonnées	5
6	Tests et validation	6
7	Conclusion	6

1 Introduction

Dans le cadre du module Projet Informatique 2, nous avons développé FlipTech, un jeu de cartes stratégique programmé en langage C.

L'objectif principal du projet était de concevoir une application fonctionnelle, stable et structurée, tout en respectant plusieurs contraintes imposées : absence de variables globales, séparation claire entre les différentes couches du programme et limitation de la taille des fonctions.

Ce projet nous a permis de mettre en pratique les notions étudiées durant l'année, notamment l'organisation modulaire d'un programme, la gestion des structures de données ainsi que le travail collaboratif autour d'un projet logiciel.

Les principales contraintes du projet concernaient la modularité du code, la robustesse des entrées utilisateur et l'absence de variables globales.

2 Architecture logicielle

2.1 Organisation du programme

Afin de faciliter le développement et la maintenance du projet, le programme a été découpé en plusieurs modules indépendants :

- `deck` : gestion et génération des cartes
- `player` : gestion des joueurs et de leurs données
- `rules` : application des règles du jeu
- `game` : déroulement des manches et boucle principale
- `ui` : affichage et interactions avec l'utilisateur
- `save/stats` : sauvegarde des données et statistiques
- `utils` : fonctions utilitaires

Chaque module possède son propre fichier d'en-tête afin de limiter les dépendances entre les différentes parties du programme et rendre le code plus lisible.

2.2 Gestion mémoire

Nous avons choisi d'utiliser uniquement des structures statiques à taille fixe :

- Deck : 85 cartes maximum
- Main d'un joueur : 20 cartes maximum

L'allocation dynamique n'a pas été utilisée afin d'éviter les problèmes liés aux pointeurs et aux fuites mémoire. Ce choix a permis d'obtenir un programme plus stable et plus simple à déboguer.

L'utilisation de structures statiques a permis de simplifier la gestion mémoire tout en garantissant un comportement prévisible du programme.

2.3 Refactorisation du code

Au début du développement, une partie importante de la logique du jeu se trouvait directement dans la boucle principale, ce qui rendait le programme difficile à maintenir.

Nous avons ensuite réorganisé le projet en centralisant les traitements dans la fonction `rules_apply_card`, devenue le point principal de gestion des règles du jeu.

Cette refactorisation a amélioré la lisibilité du code et facilité les tests.

3 Organisation du travail

Le travail a été réparti entre les différents membres du groupe afin de permettre un développement parallèle :

- Brahim : gestion du deck et génération aléatoire
- Kais : logique du jeu et moteur principal
- Youssef : interface utilisateur et système de sauvegarde

Le projet a été centralisé avec Git afin de faciliter le partage du code et les mises à jour entre les membres du groupe.

Des phases de tests communes ont également été réalisées régulièrement afin de vérifier la compatibilité entre les différents modules.

4 Détails d'implémentation

4.1 Module Deck

Le deck est généré au lancement de la partie puis mélangé à l'aide de l'algorithme de Fisher-Yates.

Une vérification supplémentaire permet de détecter lorsque la pioche est vide afin d'éviter des erreurs lors du déroulement du jeu.

4.2 Module Rules

Le module `Rules` constitue la partie centrale du programme. Il prend en charge :

- l'ajout des cartes dans la main du joueur
- la détection des doublons
- la gestion des bonus
- les conditions de victoire et de défaite

Ce module retourne un état de jeu standardisé utilisé ensuite par le module `Game` pour gérer la suite de la manche.

4.3 Interface utilisateur

Au début du projet, les saisies utilisateur étaient réalisées avec `scanf`. Cependant, plusieurs problèmes liés au buffer et aux entrées invalides rendaient le programme instable.

Nous avons donc remplacé cette méthode par `fgets` accompagné d'une validation manuelle des données saisies.

L'interface utilise également des codes ANSI afin d'améliorer la lisibilité dans le terminal et distinguer certains effets visuels liés aux cartes bonus.

4.4 Système de sauvegarde

Les données des joueurs sont enregistrées dans un fichier texte sous le format suivant :

```
nom | score total | score manche
```

Ce choix permet d'obtenir un système simple, lisible et facilement exploitable.

5 Difficultés rencontrées

5.1 Gestion des cas limites

Plusieurs situations particulières ont nécessité des corrections durant le développement :

- gestion des joueurs éliminés simultanément
- traitement d'une pioche vide
- correction de doublons détectés trop tardivement

Ces cas ont demandé plusieurs phases de tests avant d'obtenir un comportement stable.

5.2 Gestion des entrées utilisateur

La gestion des saisies utilisateur a représenté une difficulté importante du projet.

Les premières versions utilisant `scanf` provoquaient parfois des comportements imprévus ou des blocages du programme. Le passage à `fgets` avec validation manuelle a permis de corriger la majorité de ces problèmes.

Une part importante du travail de correction a concerné la sécurisation des entrées utilisateur et la gestion des cas invalides.

5.3 Fonctionnalités abandonnées

Certaines fonctionnalités prévues au départ n'ont finalement pas été intégrées afin de privilégier la stabilité générale du programme :

- intelligence artificielle basique
- nouvelles cartes spéciales
- sauvegarde complète d'une partie en cours

6 Tests et validation

Cas testé	Résultat obtenu
Détection d'un doublon	Défaite immédiate du joueur
7 cartes différentes	Victoire validée (+ 15 points)
Pioche vide	Fin correcte de la partie
Bonus cumulés	Calcul validé correctement
Entrées invalides	Nouvelle saisie demandée sans crash

Les tests ont été effectués sur plusieurs configurations de parties allant de 2 à 4 joueurs. Aucun problème critique n'a été observé dans la version finale.

7 Conclusion

Le projet FlipTech nous a permis d'appliquer concrètement les notions de programmation vues en cours à travers le développement d'un projet complet en langage C.

Ce travail nous a également confrontés à plusieurs problématiques importantes, notamment la modularité du code, la gestion des cas limites et la robustesse des interactions utilisateur.

Au final, le programme développé répond aux objectifs fixés au début du projet et constitue une application stable, structurée et fonctionnelle.

Brahim BENALI
Kais KHEMIRI
Youssef FARSAN
22 mai 2026